

Aula 07 — Processos (parte 2): Virtualização e Clientes

Resumo 8 de 14 · arquivo aula07 · 18 imagens / 17 slides · Cap. 3 de van Steen & Tanenbaum

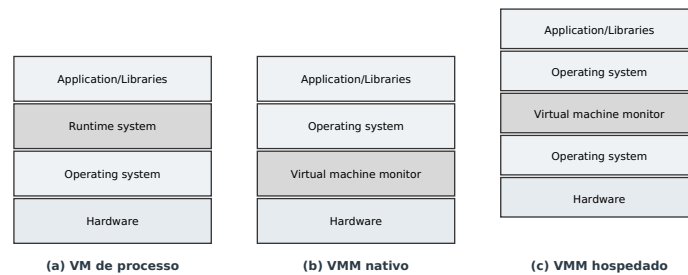
Duas metades: **virtualização** (imitar interfaces para rodar software isolado, base de VMs, containers e da nuvem) e **clientes** (como o lado do cliente é organizado — interfaces em rede, o sistema X Window e o software cliente que provê transparência de distribuição).

1. Virtualização: por quê e como

A virtualização é importante porque o **hardware muda mais rápido que o software**, facilita **portabilidade/migração** de código e permite **isolar** componentes com falha ou sob ataque. O princípio é sempre **imitar interfaces**. Há quatro tipos de interface em três níveis:

- **ISA (conjunto de instruções)**: dividido em instruções *privilegiadas* (só o SO executa) e *gerais* (qualquer programa).
- **Chamadas de sistema (syscalls)** oferecidas pelo SO.
- **Chamadas de biblioteca (API)**.

Formas de virtualização



(a) VM de processo: interpretador/emulador (runtime) sobre um SO. (b) VMM nativo: monitor direto sobre o hardware, com SO mínimo. (c) VMM hospedado: monitor delega a um SO completo por baixo.

Condição para virtualizar

Sobre as instruções: uma **privilegiada** causa uma *trap* (armadilha) para o SO se for executada em modo usuário. As **sensíveis** são as perigosas: **sensível ao controle** (pode alterar a configuração da máquina) e **sensível ao comportamento** (o efeito depende do contexto, ex.: P0PF). A **condição de virtualização** (Popek & Goldberg): é possível construir um VMM se o conjunto de instruções **sensíveis for subconjunto das privilegiadas**. Quando isso não vale (há instruções sensíveis não privilegiadas que rodam em modo usuário sem trap), as saídas são: **emular tudo**, **encapsular** as sensíveis não privilegiadas para desviar ao VMM, ou **paravirtualização** (modificar o SO convidado para eliminar/neutralizar essas instruções).

Containers

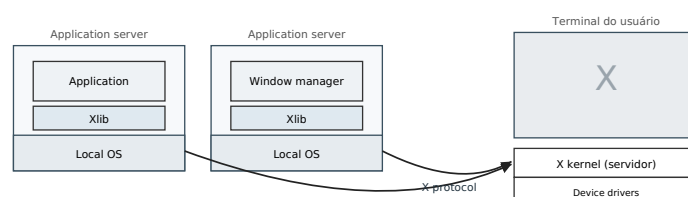
Alternativa mais leve às VMs, apoiada em três recursos do SO: **namespaces** (cada container vê seus próprios identificadores), **union file system** (combina sistemas de arquivos em camadas, com escrita só na camada superior) e **control groups (cgroups)** (impõem limites de recursos a um grupo de processos). Exemplo histórico: o **PlanetLab**, onde várias organizações doam máquinas compartilhadas para experimentos; para as aplicações não se atrapalharem, cada uma roda em *vservers* (ambientes isolados, tipo chroot), e o conjunto de vservers de uma aplicação forma um *slice*.

VMs e nuvem

A nuvem oferece três níveis de serviço: **IaaS** (infraestrutura), **PaaS** (nível de sistema/plataforma) e **SaaS** (aplicações). No IaaS, em vez de alugar uma máquina física, aluga-se uma **VM** que pode compartilhar a máquina física com outros clientes — dando isolamento quase completo (embora o *isolamento de desempenho* nem sempre seja garantido).

2. Clientes

Ao organizar o lado do cliente, distinguem-se soluções em **nível de aplicação** e em **nível de middleware**. Um caso clássico de interface de usuário em rede é o **sistema X Window**:



X Window: os aplicativos (nas “application servers”) usam a Xlib e falam pelo **protocolo X** com o **X kernel** do terminal. Detalhe contraintuitivo: o **aplicativo é o cliente** e o **X-kernel** (na máquina do usuário) é o **servidor**.

Melhorando o X: na prática nem sempre há separação clara entre a lógica da aplicação e os comandos de interface, e as aplicações operam de forma muito síncrona com o kernel X. Duas abordagens alternativas: deixar a aplicação controlar o display até **nível de pixel** (ex.: VNC) ou oferecer só **operações de alto nível** (dependentes dos drivers locais), o que é mais eficiente.

Ambiente de desktop virtual: com cada vez mais aplicações na nuvem, a pergunta é como usá-las a partir do ambiente do usuário. A resposta prática foi usar o **navegador web** como interface de usuário universal e transparente (ex.: o Google Chromebook, cujo “desktop” é essencialmente o navegador).

Software do lado do cliente (transparência de distribuição)

Boa parte do software cliente existe para dar **transparência de distribuição**:

- **Acesso:** *stubs* do lado do cliente para RPC (escondem a serialização e a chamada remota).
- **Localização/migração:** o software cliente acompanha onde está o servidor.
- **Replicação:** o stub do cliente trata múltiplas invocações às réplicas.
- **Falha:** frequentemente só dá para mascarar no cliente (tenta esconder falhas de servidor e de comunicação).